

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Môn học: Nhập môn Trí tuệ Nhân tạo

BÁO CÁO ĐỒ ÁN LAB 1

Tối ưu hóa Tuyến đường Giao hàng Đô thị TP. Hồ Chí Minh trong Điều
kiện Kẹt xe và Ngập nước (FloodRoute HCMC)

Giảng viên hướng dẫn:
TS. Nguyễn Văn Tân

Sinh viên thực hiện:
Ngô Quang Thắng -- 23127473
Vũ Lê Trọng Văn -- 20127095
Nguyễn Duy Khang -- 23127202
Lưu Huy Minh Quang -- 23127016

TP. HỒ CHÍ MINH -- THÁNG 08/2026

MỤC LỤC

1. Giới thiệu Nhóm & Phân công Đóng góp	1
2. Bối cảnh Đề tài Thực tế	2
3. Mô hình hóa Bài toán & Hàm Chi phí	3
4. Tập Dữ liệu & Quy trình Quản lý	4
5. Cơ sở Lý thuyết & Nguyên lý Thuật toán	5
6. Kiến trúc Hệ thống & Luồng Thực thi	6
7. So sánh Thuật toán & Kết quả Thực nghiệm	7
8. Tối ưu hóa Tuyến đường Giao hàng Đa điểm (TSP)	8
9. Hướng dẫn Cài đặt & Sử dụng Ứng dụng	9
10. Hạn chế & Hướng Phát triển Tương lai	10
Tài liệu Tham khảo	14

Giới thiệu Nhóm \& Phân công Đóng góp

Thông tin Nhóm

- **Tên nhóm:** FloodRoute Team
- **Trello Board Quản lý Dự án:** [AI_Project_1 \(https://trello.com/b/AGK7CP4q/aiproject1\)](https://trello.com/b/AGK7CP4q/aiproject1)

Bảng Phân công Trách nhiệm \& Đóng góp Chi tiết

Dựa trên hệ thống thẻ công việc và lịch sử commit thực tế từ Trello Board dự án:

Thành viên	Tài khoản / MSSV	Thẻ Trello	Đóng góp \	Trách nhiệm chi tiết	Tỷ lệ
Lưu Huy Minh Quang	lhmquang232 23127016	Card BE-01 Card BE-02 Card BE-03	• BE-01: Thiết kế mô hình dữ liệu bất biến Graph , Node , Edge , GraphView ; kiểm soát đồ thị có hướng, deterministic neighbor sorting, phát hiện chu trình. • BE-02: Xây dựng ScenarioCostEngine , 3 kịch bản BALANCED , PEAK_TRAFFIC , RAIN_SAFE , mô hình chi phí 4 thành phần, breakdown chi tiết từng cạnh và xử lý đóng cạnh. • BE-03: Xây dựng FastAPI Router (/locations , /scenarios , /search , /compare), validate schema Pydantic, xử lý lỗi chuẩn và sinh giải thích đường đi.	25%	
Vũ Lê Trọng Văn	vuletrngvan 20127095	Card FE-01 Card FE-02 Card OPT	• FE-01: Xây dựng API client TypeScript, hệ thống controls chọn Start/Goal/Algorithm/Scenario, tích hợp loading/error state và phím tắt. • FE-02: Tích hợp MapLibre GL JS, hiển thị đồ thị qua GPU layers, phát triển Trace Player (Play/Pause/Step/Speed) và Dashboard hiển thị metrics, explanation. • OPT: Giải bài toán giao hàng đa điểm (TSP): xây dựng PairwiseMatrix , giải thuật chính xác Held-Karp DP ($O(N^2 \cdot 2^N)$), giải thuật xấp xỉ Nearest Neighbor, đo % Approximation Gap và % Baseline Savings.	25%	
Nguyễn Duy Khang	11nguynduykhang 23127202	Card ALG-01 Card ALG-03 Card QA-01	• ALG-01: Cài đặt thuật toán BFS (Queue) và DFS (Stack) với cơ chế deterministic tie-breaking, ghi nhận trace, và xây dựng phân ví dụ chứng minh giới hạn của BFS/DFS. • ALG-03: Cài đặt Greedy Best-First Search và Bidirectional Search, xác định điểm giao nhau và chốt điều kiện dừng, viết ADR Bidirectional Search. • QA-01: Xây dựng bộ kiểm thử so sánh thuật toán, golden regression test suite, kịch bản demo và kiểm thử tính hợp lệ của hệ thống.	25%	
Ngô Quang Thắng	tartzuser 23127473	Card ALG-02 Card DATA-01 Card DATA-02	• ALG-02: Cài đặt thuật toán UCS và A* Search, phát triển hàm Heuristic Haversine có trọng số, chứng minh toán học tính Admissible và Consistent của heuristic. • DATA-01: Xây dựng pipeline dữ liệu OSM 4 lớp (raw → interim → processed → fixtures), quản lý metadata manifest và SHA-256 checksums. • DATA-02: Map-match danh mục 24 điểm ngập lụt TP. Thủ Đức vào đồ thị, thiết lập kịch bản triều cường/mưa lớn, kiểm định chất lượng dữ liệu (QC) và gán nhãn provenance.	25%	

Bảng: Bảng phân công trách nhiệm và đóng góp của các thành viên trong nhóm.

Bảng Tự Đánh giá Mức độ Hoàn thành Mục tiêu Đề cương (Rubric Matrix)

Tiêu chí Đánh giá (Rubric)	Điểm	Đánh giá	Minh chứng mã nguồn \	Kết quả thực tế
1. Bối cảnh giao thông Việt Nam \	Kịch bản thực tế	10	10/10	Bối cảnh shipper giao hàng TP. Thủ Đức, kẹt xe giờ cao điểm và ngập nước triều cường/mưa lớn thực tế.
2. Mô hình đồ thị, tập dữ liệu \	Hàm chi phí	15	15/15	Đồ thị có hướng bất biến, 3 dataset (Landmark 65, Market 90, Capacity 3229), chi phí tổng hợp 4 thành phần ($D, T_{ff}, P_{traffic}, R_{flood}$).
3. Cài đặt các thuật toán bắt buộc (BFS, DFS, UCS, A*)	20	20/20	4 thuật toán chuẩn với deterministic tie-break, sinh trace từng bước, chứng minh toán học Heuristic Haversine Admissible/Consistent.	
4. Cài đặt các thuật toán mở rộng (Greedy, Bidirectional)	10	10/10	Cài đặt Greedy Best-First Search và Bidirectional Search trên đồ thị có hướng chuyển vị; kèm phân ví dụ và ADR.	
5. Bài toán tối ưu hóa nhiều địa điểm (TSP)	10	10/10	Xây dựng Pairwise Matrix có hướng, giải thuật chính xác Held-Karp DP ($O(N^2 \cdot 2^N)$), Nearest Neighbor, đo % Gap và % Baseline Savings.	
6. Giao diện người dùng (GUI) \	Trực quan hóa tiến trình	10	10/10	Web app React + MapLibre GL JS vector tiles, GPU layers, snap 200m, Trace Player (Frontier/Current/Visited), i18n EN/VI, Responsive.
7. Giải thích tuyến đường \	So sánh phương án	10	10/10	Tự động sinh diễn giải ngôn ngữ tự nhiên: phân tích thành phần chi phí chiếm ưu thế, chỉ ra đoạn đường gây penalty lớn nhất, so sánh shortest vs fastest vs best cost.
8. Chất lượng Báo cáo Kỹ thuật	10	10/10	Đầy đủ 10 chương mục (a-j), trình bày học thuật, số liệu thực nghiệm từ 3.600 lượt chạy benchmark, công thức toán và sơ đồ rõ ràng.	
9. Chất lượng Video Demo	5	5/5	Video demo trực quan đầy đủ các tính năng, giải thích thuật toán từng bước qua trace player và so sánh kịch bản.	
TỔNG CỘNG	100	100/100	Dự án hoàn thành toàn diện 100% các tiêu chí kỹ thuật.	

Bảng: Ma trận tự đánh giá mức độ hoàn thành theo chuẩn Rubric Lab 1.

Bối cảnh Đề tài Thực tế

Thực trạng Giao thông Đô thị tại TP. Hồ Chí Minh và TP. Thủ Đức

Tại các đô thị lớn của Việt Nam, đặc biệt là TP. Hồ Chí Minh, giao thông đô thị có mật độ phương tiện xe máy rất cao kết hợp với mạng lưới hẻm và đường một chiều dày đặc. Vào các khung giờ cao điểm (07:00 -- 08:30 sáng và 16:30 -- 18:30 chiều), các trục đường huyết mạch khu vực TP. Thủ Đức như **Võ Văn Ngân, Kha Vạn Cân, Đặng Văn Bi, Tô Ngọc Vân, Quốc lộ 13** thường xuyên rơi vào tình trạng ùn ứ nghiêm trọng, làm gia tăng thời gian di chuyển gấp 2--4 lần so với điều kiện lưu thông tự do (*Free-Flow*). **Hình:** Đặc thù giao thông đô thị kẹt tại khu vực nghiên cứu TP. Thủ Đức.

Vấn đề Ngập nước do Triều cường và Mưa lớn

Đặc thù địa hình trũng thấp và hệ thống thoát nước quá tải khiến khu vực xung quanh **Chợ Thủ Đức (đường Dương Văn Cam, Đặng Thị Rành, Kha Vạn Cân, Hồ Văn Tư)** trở thành "rốn ngập" kinh niên mỗi khi có mưa lớn hoặc triều cường dâng cao (mức nước ngập thường từ 0.3m đến 0.6m). Đối với người giao hàng bằng xe máy:

- Việc đi vào đường ngập sâu gây chết máy, hư hỏng hàng hóa và nguy cơ tai nạn nghiêm trọng.
- Tuyến đường ngắn nhất về mặt khoảng cách địa lý (D) nếu băng qua rốn ngập sẽ trở thành **tuyến đường bất khả thi** hoặc có chi phí rủi ro cực cao.

Mục tiêu của Ứng dụng FloodRoute HCMC

Hệ thống **FloodRoute HCMC** giải quyết triệt để hai bài toán thực tế:

1. **Tìm đường 2 điểm thích ứng động:** Không chỉ dựa trên cự ly hình học, mà kết hợp đồng thời khoảng cách, thời gian lưu thông, mức độ kẹt xe và nguy cơ ngập lụt để đề xuất lộ trình an toàn, tối ưu nhất.
2. **Tối ưu hóa hành trình giao hàng đa điểm (TSP):** Cho phép shipper nhập 1 kho hàng (*depot*) và 5--10 điểm giao hàng (*stops*), từ đó giải thuật tự động sắp xếp thứ tự giao hàng tối ưu nhằm giảm thiểu tổng chi phí, thời gian và nhiên liệu.

Mô hình hóa Bài toán \& Hàm Chi phí

Mô hình Không gian Trạng thái (State-Space Formulation)

Bài toán tìm đường được mô hình hóa toán học dưới dạng bài toán tìm kiếm trên không gian trạng thái:

- **Tập trạng thái (S):** Tập hợp tất cả các nút giao thông và địa điểm trọng yếu trên mạng lưới đường bộ ($S = V$).
- **Trạng thái bắt đầu (s_0):** Vị trí xuất phát của shipper ($s_0 = \text{Start} \in V$).
- **Tập hành động ($A(s)$):** Tập các cạnh có hướng xuất phát từ nút s mà không bị đóng trong kịch bản hiện tại:

$$A(s) = \{(s, v) \in E \mid (s, v) \notin \text{ClosedEdges}(\text{Scenario})\}$$

- **Mô hình chuyển trạng thái ($\text{Result}(s, a)$):** Khi thực hiện hành động $a = (s, v)$, trạng thái mới đạt được là v .
- **Kiểm tra trạng thái đích ($\text{GoalTest}(s)$):** Kiểm tra xem s có trùng với điểm đích **Goal** hay không ($s = \text{Goal}$).
- **Chi phí từng bước ($c(s, a, s')$):** Chi phí tổng hợp của cạnh $a = (s, s')$ tính theo mô hình chi phí của kịch bản đang chọn.

Mô hình Chi phí Tổng hợp 4 Thành phần (Composite Cost Function)

Chi phí của mỗi cạnh có hướng $e = (u, v)$ được tính toán thông qua hàm tuyến tính có trọng số chuẩn hóa:

$$C(e) = w_{\text{dist}} \cdot D(e) + w_{\text{time}} \cdot T_{\text{ff}}(e) + w_{\text{cong}} \cdot P_{\text{traffic}}(e) + w_{\text{flood}} \cdot R_{\text{flood}}(e)$$

Trong đó:

1. $D(e)$ -- Khoảng cách (Distance): Độ dài vật lý của đoạn đường, quy đổi ra kilômét:

$$D(e) = \frac{\text{distance_m}(e)}{1000} \quad (\text{km})$$

2. $T_{\text{ff}}(e)$ -- Thời gian lưu thông tự do (Free-flow Time): Thời gian di chuyển ở điều kiện đường thông thoáng với vận tốc thiết kế:

$$T_{\text{ff}}(e) = \text{free_flow_time_min}(e) \quad (\text{phút})$$

3. $P_{\text{traffic}}(e)$ -- Độ trễ do kẹt xe (Traffic Delay Penalty):

$$\begin{aligned} T_{\text{travel}}(e) &= T_{\text{ff}}(e) \times \text{TrafficMultiplier}(e) \\ P_{\text{traffic}}(e) &= \max(0, T_{\text{travel}}(e) - T_{\text{ff}}(e)) \quad (\text{phút}) \end{aligned}$$

Lưu ý thiết kế: Việc tách riêng P_{traffic} khỏi T_{ff} giúp chi phí không bị tính trùng lặp thời gian di chuyển cơ sở.

4. $R_{\text{flood}}(e)$ -- Chỉ số Rủi ro Ngập nước (Flood Risk): Giá trị chuẩn hóa $R_{\text{flood}} \in [0.0, 1.0]$ dựa trên mức độ ngập lịch sử (0: khô ráo, 1: ngập sâu nguy hiểm).
5. Bộ trọng số ($w_{\text{dist}}, w_{\text{time}}, w_{\text{cong}}, w_{\text{flood}}$): Thỏa mãn điều kiện chuẩn hóa $w_i \geq 0$ và:

$$\sum w_i = w_{\text{dist}} + w_{\text{time}} + w_{\text{cong}} + w_{\text{flood}} = 1.0$$

Các Thiết lập Kịch bản Chuẩn (Cost Presets)

Preset ID	w_{dist}	w_{time}	w_{cong}	w_{flood}	Mục tiêu Tối ưu hóa
BALANCED	0.25	0.30	0.20	0.25	Cân bằng hài hòa giữa cự ly, thời gian, kẹt xe và an toàn ngập nước (chế độ mặc định).
PEAK_TRAFFIC	0.15	0.25	0.45	0.15	Ưu tiên cao nhất cho việc tránh các điểm nóng kẹt xe trong giờ cao điểm.
RAIN_SAFE	0.10	0.25	0.15	0.50	Ưu tiên tối đa việc đi vòng qua các tuyến đường ngập sâu khi trời mưa to / triều cường.

Bảng: Các bộ trọng số kịch bản chuẩn được tích hợp sẵn trong hệ thống.

Tập Dữ liệu & Quy trình Quản lý

Quy trình Xử lý Dữ liệu 4 Tầng (4-Tier Data Pipeline)

Dự án áp dụng quy trình quản trị dữ liệu 4 tầng nghiêm ngặt nhằm đảm bảo tính toàn vẹn và khả năng tái lập:

Hình: Quy trình xử lý dữ liệu 4 tầng của FloodRoute HCMC.

- **data/raw/**: Chứa dữ liệu gốc bất biến (`osm_relation_19407794.geojson`, snapshot Kaggle UTraffic 8.952 nodes/14.043 edges), kiểm tra toàn vẹn bằng mã SHA-256.
- **data/interim/**: Lưu trữ dữ liệu trung gian sau khi trích lọc theo Bounding Box địa lý.
- **data/processed/**: Các graph release phục vụ chạy ứng dụng và thực nghiệm khoa học.
- **data/fixtures/**: Các đồ thị nhỏ phục vụ unit test và golden validation (`toy_graph_v0.1`).

Chi tiết 3 Bản Phát hành Dữ liệu (Processed Releases)

Tên Dataset	Số Đỉnh (<i>V</i>)	Số Cạnh (<i>E</i>)	Trạng thái	Mục đích Sử dụng
thu_duc_landmarks_v1.0.0	65 POI	178 Cạnh	LANDMARK_DEMO	65 địa điểm lớn có tên thật từ OSM (Đại học, Bệnh viện, Chợ); cạnh lưu polyline thực tế từ UTraffic. Mặc định trên GUI.
thu_duc_market_v1.0.0	90 Nút	155 Cạnh	REVIEW_REQUIRED	Vùng trọng điểm Chợ Thủ Đức; tích hợp 3 kịch bản thời tiết và 24 điểm ngập phục vụ demo đổi tuyến.
thu_duc_core_capacity_v0.1.0	3.229 Nút	5.057 Cạnh	CAPACITY_ONLY	Mạng lưới giao thông lõi TP. Thủ Đức phục vụ đo đặc hiệu năng và stress-test dung lượng.

Bảng: Tổng hợp 3 bản phát hành dữ liệu chính thức trong hệ thống.

Hệ thống Nhãn Dữ liệu Minh bạch (Provenance Transparency)

- **SOURCE_BACKED**: Dữ liệu có nguồn trích xuất xác thực từ OpenStreetMap hoặc UTraffic.
- **DERIVED**: Dữ liệu tính toán từ công thức/script tái lập (cụ ly Haversine, Free-flow time).
- **SIMULATED / ASSUMPTION**: Dữ liệu mô phỏng tham số giao thông (hiển thị rõ trên GUI).
- **REVIEW_REQUIRED**: Dữ liệu map-matching ngập nước đang chờ thẩm định bổ sung.

Cơ sở Lý thuyết & Nguyên lý Thuật toán

Thuật toán Tìm kiếm Không Trọng số (Unweighted Search)

- **Breadth-First Search (BFS)**: Sử dụng hàng đợi FIFO (`collections.deque`). Khám phá đều theo từng mức độ sâu; đảm bảo tìm được đường đi có **số chặng ít nhất** (*minimum hop count*).
- **Depth-First Search (DFS)**: Sử dụng ngăn xếp LIFO kết hợp tập `expanded` để chống lặp chu trình. Khám phá sâu dọc theo từng nhánh đường trước khi quay lui (*backtracking*); không đảm bảo tính tối ưu chi phí.

Thuật toán Tìm kiếm Có Trọng số (Weighted Search)

- **Uniform Cost Search (UCS / Dijkstra)**: Sử dụng Min-Priority Queue (`heapq`) với khóa sắp xếp là $g(n)$ (chi phí tích lũy từ Start đến n). Đảm bảo tìm được đường đi có **tổng chi phí C^* nhỏ nhất** khi mọi $c(e) \geq 0$.
- **A* Search**: Sử dụng Min-Priority Queue với hàm đánh giá:

$$f(n) = g(n) + h(n)$$

với hàm Heuristic khoảng cách Haversine có trọng số:

$$h(n) = w_{\text{dist}} \cdot D_{\text{Haversine}}(n, \text{Goal})$$

Chứng minh Toán học về Tính Đúng đắn của Heuristic

Định lý (Tính Chấp nhận được -- Admissibility)

Hàm heuristic $h(n) = w_{\text{dist}} \cdot D_{\text{Haversine}}(n, \text{Goal})$ luôn thỏa mãn $0 \leq h(n) \leq h^*(n)$ với mọi nút n , trong đó $h^*(n)$ là chi phí thực tế tối ưu từ n đến Goal .

Chứng minh:

1. Khoảng cách đường cong trắc địa (đường chim bay) $D_{\text{Haversine}}(n, \text{Goal})$ là khoảng cách vật lý ngắn nhất giữa hai điểm trên bề mặt Trái Đất. Do đó, với bất kỳ đường đi thực tế nào qua mạng lưới đường bộ, tổng chiều dài $D^*(n, \text{Goal}) \geq D_{\text{Haversine}}(n, \text{Goal})$.
2. Mọi thành phần chi phí trong hàm mục tiêu ($T_{\text{ff}}, P_{\text{traffic}}, R_{\text{flood}}$) và các trọng số w_i đều không âm (≥ 0).
3. Do đó, chi phí của mỗi cạnh $c(e) \geq w_{\text{dist}} \cdot D(e)$. Tổng chi phí thực tế tối ưu thỏa mãn:

$$h^*(n) \geq w_{\text{dist}} \cdot D^*(n, \text{Goal}) \geq w_{\text{dist}} \cdot D_{\text{Haversine}}(n, \text{Goal}) = h(n)$$

$\Rightarrow h(n)$ luôn là một chặn dưới (*lower bound*) hợp lệ, do đó $h(n)$ Admissible. ■

Định lý (Tính Nhất quán / Đơn điệu -- Consistency / Monotonicity)

Với mọi cạnh chuyển trạng thái $e = (n, n')$, hàm heuristic thỏa mãn $h(n) \leq c(n, n') + h(n')$.

Chứng minh:

1. Theo bất đẳng thức tam giác cầu (*Spherical Triangle Inequality*) cho khoảng cách địa lý:

$$D_{\text{Haversine}}(n, \text{Goal}) \leq D_{\text{Haversine}}(n, n') + D_{\text{Haversine}}(n', \text{Goal})$$

2. Nhân hai vế với trọng số không âm $w_{\text{dist}} \geq 0$:

$$w_{\text{dist}} \cdot D_{\text{Haversine}}(n, \text{Goal}) \leq w_{\text{dist}} \cdot D_{\text{Haversine}}(n, n') + w_{\text{dist}} \cdot D_{\text{Haversine}}(n', \text{Goal})$$

$$\Leftrightarrow h(n) \leq w_{\text{dist}} \cdot D_{\text{Haversine}}(n, n') + h(n')$$

3. Vì chiều dài đoạn đường thực tế $D(n, n') \geq D_{\text{Haversine}}(n, n')$ và chi phí cạnh $c(n, n') \geq w_{\text{dist}} \cdot D(n, n')$, ta có:

$$w_{\text{dist}} \cdot D_{\text{Haversine}}(n, n') \leq c(n, n')$$

4. Kết hợp lại suy ra $h(n) \leq c(n, n') + h(n') \quad \forall (n, n') \in E \Rightarrow h(n)$ Consistent. ■

Thuật toán Tìm kiếm Mở rộng

- **Greedy Best-First Search:** Sử dụng hàng đợi ưu tiên với hàm đánh giá $f(n) = h(n)$. Tốc độ thực thi cực nhanh nhưng không đảm bảo tính tối ưu.
- **Bidirectional Search:** Duyệt đồng thời hai luồng BFS: luồng thuận (*Forward*) từ **Start** và luồng nghịch (*Backward*) từ **Goal** trên đồ thị chuyển vị. Giảm không gian tìm kiếm từ $O(b^d)$ xuống $O(2 \cdot b^{d/2})$.

Ví dụ Minh họa Từng bước Duyệt Nút (Trace Walkthrough Example)

Xét đồ thị mẫu 4 nút: $A \xrightarrow{c=2, h=5} B \xrightarrow{c=4, h=0} D$ và $A \xrightarrow{c=5, h=3} C \xrightarrow{c=1, h=0} D$ với **Start** = *A*, **Goal** = *D*.

Bước	Thuật toán	Nút EXPAND	Frontier List (Nút, g, h, f)	Nút CLOSE	Ghi chú Trạng thái
1	UCS	<i>A</i> ($g = 0$)	$[(B, g = 2), (C, g = 5)]$	$\{A\}$	Mở rộng <i>A</i> , phát hiện <i>B</i> và <i>C</i> .
2	UCS	<i>B</i> ($g = 2$)	$[(C, g = 5), (D, g = 6)]$	$\{A, B\}$	<i>B</i> có $g = 2$ nhỏ nhất. Thêm <i>D</i> ($g = 6$).
3	UCS	<i>C</i> ($g = 5$)	$[(D, g = 6)_{\text{qua } C}, (D, g = 6)_{\text{qua } B}]$	$\{A, B, C\}$	<i>C</i> có $g = 5$. Thêm <i>D</i> qua <i>C</i> ($g = 6$).
4	UCS	<i>D</i> ($g = 6$)	Đích đạt được	$\{A, B, C, D\}$	Nghiệm tối ưu: $A \rightarrow B \rightarrow D$ ($C = 6.0$).
1	A*	<i>A</i> ($f = 5$)	$[(B, g = 2, h = 5, f = 7), (C, g = 5, h = 3, f = 8)]$	$\{A\}$	<i>A</i> khởi tạo với $h(A) = 5$.
2	A*	<i>B</i> ($f = 7$)	$[(D, g = 6, h = 0, f = 6), (C, g = 5, h = 3, f = 8)]$	$\{A, B\}$	<i>B</i> có $f = 7 < f(C) = 8$. Thêm <i>D</i> ($f = 6$).
3	A*	<i>D</i> ($f = 6$)	Đích đạt được	$\{A, B, D\}$	A* dừng sớm, không cần duyệt <i>C</i> !

Bảng: Bảng so sánh quá trình duyệt nút từng bước giữa UCS và A* trên đồ thị mẫu.

Kiến trúc Hệ thống & Luồng Thực thi

Sơ đồ Kiến trúc Phân lớp (Clean Architecture)

Hệ thống được thiết kế theo mô hình phân lớp rõ ràng, đảm bảo tính độc lập giữa giao diện người dùng, API và động cơ thuật toán:

Hình: Kiến trúc phân tầng Clean Architecture của FloodRoute HCMC.

Luồng Xử lý Tìm kiếm 2 Điểm (Two-Point Search Workflow)

Khi người dùng thực hiện tìm đường trên giao diện:

1. **Khởi tạo Yêu cầu:** Người dùng chọn điểm Start, Goal (qua click map hoặc tìm kiếm), chọn Kịch bản (vd: Mưa lớn) và Thuật toán (vd: A*).
2. **Gửi API Request:** React gửi request `POST /api/v1/search` đến FastAPI Backend.

3. **Xác thực \& Điều phối:** SearchService nạp graph từ catalog, áp dụng kịch bản qua GraphView và gọi hàm thuật toán tương ứng.
4. **Tính toán \& Ghi nhận Trace:** Thuật toán duyệt đồ thị, tính chi phí cạnh qua ScenarioCostEngine và sinh danh sách TraceEvent (OPEN, EXPAND, RELAX, CLOSE, GOAL).
5. **Giải thích Tuyến đường:** ExplanationEngine phân tích thành phần chi phí chiếm ưu thế và xác định đoạn đường có penalty kẹt xe/ngập lớn nhất.
6. **Hiển thị Thực quan:** React vẽ đường đi tối ưu lên bản đồ vector và kích hoạt Trace Player cho phép người dùng xem lại toàn bộ quá trình mở rộng nút.

So sánh Thuật toán \& Kết quả Thực nghiệm

Bảng So sánh Lý thuyết Toàn diện giữa 6 Thuật toán

Tiêu chí	BFS	DFS	UCS	A* Search	Greedy	Bidirectional
Cấu trúc	Queue	Stack	Min-Heap	Min-Heap (<i>g + h</i>)	Min-Heap (<i>h</i>)	2 Queues
Thời gian	$O(b^d)$	$O(b^m)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^d)$	$O(b^m)$	$O(2 \cdot b^{d/2})$
Bộ nhớ	$O(b^d)$	$O(bm)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^d)$	$O(bm)$	$O(2 \cdot b^{d/2})$
Đầy đủ	Có	Có (hữu hạn)	Có ($\epsilon \geq 0$)	Có (<i>h</i> Admissible)	Không	Có
Tối ưu	Số chẵn	Không	Chi phí <i>C*</i>	Chi phí <i>C*</i>	Không	Số chẵn
Heuristic	Không	Không	Không	Bắt buộc	Bắt buộc	Không

Bảng: Bảng so sánh cơ sở lý thuyết giữa 6 thuật toán tìm kiếm.

Kết quả Thực nghiệm Khoa học từ 3.600 Lượt chạy Benchmark

Dữ liệu được trích xuất trực tiếp từ artifact thực nghiệm chuẩn experiments/results/search_benchmark_v1.json (10 cặp O-D × 3 kịch bản × 6 thuật toán × 20 lần lặp độc lập):

Thuật toán	Node duyệt	Node (Med)	Thời gian (Med)	P95 (ms)	Chi phí <i>C*</i>	Cự ly (km)	ETA (p)
A_STAR	46.5	42.5	1.5082 ms	1.9687 ms	0.5715	0.322	0.89
UCS	49.7	47.0	1.2712 ms	1.6317 ms	0.5715	0.322	0.89
GREEDY	17.3	17.0	0.3825 ms	0.5998 ms	0.5825	0.321	0.85
BIDIRECTIONAL	27.0	22.5	0.0997 ms	0.1424 ms	0.6469	0.356	0.96
BFS	43.8	39.0	0.2053 ms	0.3059 ms	0.6469	0.356	0.96
DFS	59.7	70.0	0.2723 ms	0.3762 ms	0.5963	0.332	0.90

Bảng: Bảng tổng hợp số liệu thực nghiệm toàn cục (Global Benchmark Summary - 3.600 runs).

Phân tích Tuyến đường Thay đổi theo Kịch bản (Case Study OD03)

Xét cặp điểm O-D số OD03 kết nối xuyên qua khu vực Chợ Thủ Đức:

- Kịch bản HISTORICAL_OFFPEAK** : Đường phố thông thoáng, thuật toán chọn tuyến đường thẳng ngắn nhất xuyên qua trung tâm Chợ Thủ Đức (*D* = 0.163 km, Chi phí *C* = 0.1045, ETA 0.22 phút).

- **Kịch bản HISTORICAL_PM_PEAK**: Giao thông quanh chợ ùn tắc, thành phần P_{traffic} làm chi phí tăng lên $C = 0.1250$ (ETA tăng lên **0.31** phút).
- **Kịch bản RAIN_FLOOD_AWARE_2025_2026**: Cảnh qua đường Dương Văn Cam và Đặng Thị Rành có rủi ro ngập $R_{\text{flood}} = 1.0$. Thuật toán tự động chuyển hướng đường đi vòng qua đường vành đai cao ráo, bảo đảm phương tiện không bị chết máy.

Tối ưu hóa Tuyến đường Giao hàng Đa điểm (TSP)

Đặc tả Bài toán Giao hàng Đa điểm (TSP)

Khi shipper cần giao hàng cho N địa điểm ($5 \leq N \leq 10$) xuất phát từ 1 kho hàng (*Depot*):

- Cần xác định một chu trình ghé thăm mỗi điểm giao hàng đúng một lần rồi quay về kho sao cho tổng chi phí là nhỏ nhất.
- Do mạng lưới đô thị có nhiều tuyến đường một chiều, ma trận chi phí giữa các điểm là **Ma trận Bất đối xứng (Asymmetric Pairwise Matrix)**: $c(u, v) \neq c(v, u)$.

Hai Giải thuật Cài đặt

1. **Held-Karp Dynamic Programming (Giải thuật Chính xác)**: Sử dụng mặt nạ bit (*bitmask*) biểu diễn tập các đỉnh đã thăm:

$$dp(\text{mask}, u) = \min_{v \in \text{mask} \setminus \{u\}} [dp(\text{mask} \setminus \{u\}, v) + c(v, u)]$$

Chi phí chu trình tối ưu:

$$\text{Cost}_{\text{optimal}} = \min_{u \in \{1, \dots, N-1\}} [dp((1 \ll N) - 1, u) + c(u, 0)]$$

Độ phức tạp: Thời gian $O(N^2 \cdot 2^N)$, Bộ nhớ $O(N \cdot 2^N)$. Với $N \leq 10$, thời gian thực thi < 5 ms, đảm bảo nghiệm tối ưu tuyệt đối.

2. **Nearest Neighbor Heuristic (Giải thuật Xấp xỉ Nhanh)**: Tham lam chọn chặng tiếp theo có chi phí nhỏ nhất. Độ phức tạp: Thời gian $O(N^2)$, Bộ nhớ $O(N)$.

Đánh giá Thực nghiệm: So sánh Held-Karp vs Nearest Neighbor vs Thứ tự Gốc

Phương pháp	Lộ trình Ghé thăm (Visit Order)	Chi phí (C)	Cự ly (D)	% Gap	% Savings
Thứ tự Gốc (User Input)	Depot $\rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \rightarrow S_5 \rightarrow$ Depot	14.850	4.250 km	---	0.0% (Baseline)
Nearest Neighbor	Depot $\rightarrow S_2 \rightarrow S_1 \rightarrow S_3 \rightarrow S_5 \rightarrow S_4 \rightarrow$ Depot	11.230	3.120 km	+8.9%	24.4%
Held-Karp (Optimal)	Depot $\rightarrow S_2 \rightarrow S_3 \rightarrow S_5 \rightarrow S_4 \rightarrow S_1 \rightarrow$ Depot	10.310	2.890 km	0.0%	30.6%

Bảng: Bảng so sánh hiệu quả tối ưu hóa lộ trình giao hàng đa điểm ($N = 6$).

Kết luận: Thuật toán chính xác **Held-Karp DP** giúp shipper tiết kiệm đến **30.6 chi phí và quãng đường** so với thứ tự nhập ban đầu; **Nearest Neighbor** đạt mức tiết kiệm 24.4 với độ lệch xấp xỉ so với tối ưu chỉ 8.9.

Hướng dẫn Cài đặt & Sử dụng Ứng dụng

Yêu cầu Hệ thống & Cài đặt Môi trường

Yêu cầu phần mềm: Python 3.11+, Node.js 22.12+ và npm.

```
# 1. Khởi tạo môi trường ảo Python và cài đặt Backend
python -m venv .venv
.\venv\Scripts\python.exe -m pip install -e ".\backend[dev]"

# 2. Cài đặt các thư viện Frontend
npm --prefix frontend install

# 3. Khởi chạy đồng thời cả Backend và Frontend
npm run dev
```

Địa chỉ truy cập ứng dụng:

- **Giao diện Web UI:** <http://localhost:5173>
- **FastAPI Documentation (Swagger UI):** <http://localhost:8000/docs>

Hướng dẫn Sử dụng Giao diện Người dùng (GUI Guide)

1. **Chọn Điểm Trục quan trên Bản đồ (Auto-snapping 200m):** Người dùng có thể click trực tiếp lên bản đồ; hệ thống sẽ tự động snap vào điểm nút giao thông gần nhất trong bán kính 200m mà không làm giật camera. Điểm Start được đánh dấu bằng ghim màu cam, điểm Goal bằng ghim màu hồng.
2. **Chế độ Giao hàng Đa điểm (Multi-Stop Tour Mode):** Cho phép click chọn liên tiếp tối đa 10 điểm dừng trên bản đồ; hệ thống đánh số thứ tự trục quan và tự động tính toán lộ trình tối ưu qua Held-Karp DP.
3. **Thanh điều khiển Trace Player:** Hỗ trợ xem từng bước duyệt nút thuật toán với 3 màu phân biệt: Vàng (*frontier* -- nút đang chờ), Xanh lá (*current* -- nút đang duyệt), Xám (*closed* -- nút đã duyệt xong).
4. **Chế độ So sánh (Compare Mode):** Chạy song song cả 6 thuật toán trên cùng một cặp điểm và kích bản, hiển thị bảng so sánh trực quan đa tiêu chí.
5. **Chuyển đổi Ngôn ngữ & Responsive:** Hỗ trợ song ngữ Tiếng Việt (vi) / Tiếng Anh (en) và tự động co giãn menu ngăn kéo trên thiết bị di động/tablet.

Hạn chế & Hướng Phát triển Tương lai

Khó khăn Kỹ thuật Đã Vượt qua

- **Xử lý Đồ thị Nén nhưng Bảo toàn Hình học Tuyến đường:** Rút gọn mạng lưới giao thông phức tạp 14.043 cạnh thành 178 cạnh nối giữa 65 địa điểm lớn mà vẫn giữ nguyên tọa độ polyline khúc khuỷu của đường phố thật thay vì vẽ đường thẳng giả tạo.
- **Đảm bảo Tính Đúng đắn của Heuristic Đa Chi phí:** Chứng minh và thiết lập hàm heuristic Haversine có trọng số vừa thỏa mãn tính chấp nhận được (*Admissible*), vừa thỏa mãn tính đơn điệu (*Consistent*) trong không gian chi phí 4 thành phần.
- **Kiến trúc Bất biến & Deterministic Tie-breaking:** Đảm bảo thuật toán độc lập 100 với framework HTTP và luôn cho kết quả tái lập tuyệt đối trên mọi nền tảng.

Các Hạn chế Hiện tại của Hệ thống

1. **Dữ liệu Giao thông Lịch sử:** Hiện tại mô hình sử dụng hồ sơ giao thông lịch sử phân chia theo khung giờ (*Historical Traffic Profile*), chưa kết nối với luồng dữ liệu GPS xe máy theo thời gian thực (*Live Telemetry Feed*).
2. **Mô hình Ngập nước Tĩnh:** Kịch bản ngập dựa trên danh mục các điểm đen thống kê, chưa tích hợp mô hình mô phỏng thủy lực/thủy văn động lực học (*Hydraulic Dynamic Inundation*) theo lượng mưa tức thời.
3. **Phạm vi Bài toán TSP:** Hiện tại giải quyết tối ưu cho 1 shipper xe máy đơn lẻ (Single-vehicle TSP), chưa mở rộng cho bài toán phân phối đội xe nhiều phương tiện có ràng buộc tải trọng và khung giờ giao hàng (*Capacitated Vehicle Routing Problem with Time Windows -- CVRPTW*).

Định hướng Mở rộng Tương lai

- **Tích hợp API Bản đồ Thời gian Thực:** Kết nối Live Traffic API (Here Maps, TomTom, Vietbando) để cập nhật độ trễ thời gian thực vào P_{traffic} .
- **Ứng dụng Meta-heuristics cho Bài toán Quy mô Lớn:** Cài đặt thuật toán Di truyền (*Genetic Algorithm -- GA*) và Tối ưu hóa Đàn kiến (*Ant Colony Optimization -- ACO*) để giải bài toán VRP cho đội xe quy mô lớn ($N > 50$ điểm giao hàng).
- **Hệ thống Cảnh báo Dự báo Mưa ngập Tức thời (Nowcasting):** Kết hợp dữ liệu radar thời tiết để dự báo trước 30--60 phút các cung đường sắp ngập và tự động tái định tuyến (*dynamic rerouting*) cho shipper đang di chuyển.

Tài liệu Tham khảo

1. Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson, 2020.
2. Michael Held and Richard M. Karp. A Dynamic Programming Approach to Sequencing Problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196--210, 1962.
3. Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100--107, 1968.
4. OpenStreetMap Contributors. Planet dump retrieved from <https://planet.openstreetmap.org>, 2026.
5. UTraffic Dataset: Urban Traffic Flow and Speed Profiling in Ho Chi Minh City. Kaggle Open Dataset, 2025.